HANDOFF — Стандартотворець

> Чистий знімок стану для нового агента. Прочитай це + `git log --oneline -20` + `CONTINUATION.md` — і за 5 хвилин зрозумієш, де ми і що робити далі. Оновлюй цей файл, коли змінюється щось **фундаментальне** (стек, структура, deploy).

TL;DR (30 секунд)

Стандартотворець — платформа управління lifecycle стандартів у робочих групах (держсектор, UA). Робочі групи → стандарти → засідання → голосування → завдання → документи → аудит-лог. Веб-застосунок Next.js, multi-RBAC, темна/світла тема, кирилиця.

Етап: працює на staging/prod (Railway). Циклі hardening (B-1…B-9) і voting/documents lifecycle закриті. Останній закритий цикл — «Завдання» (підзадачі, шаблони, дерево), 2026-07-14 → 2026-08-06, **без формального QA-прогону**. Поточний стан і пріоритети — у `CONTINUATION.md`.

Технічний стек

Шар	Технологія
Runtime	Node 20+ (dev на 22), pnpm 9.15.9
Framework	Next.js 14.2.35 (App Router)
UI	React 18.3, TailwindCSS 3.4 (token-driven theming), Radix UI, lucide-react
Редактор	TipTap 3 (тіло стандарту / протоколи)
API	tRPC v11 (`@trpc/server` rc.522)
DB	PostgreSQL (Prisma 5.22, **`prisma db push`** — без migration-файлів)
Auth	NextAuth v4 (Credentials + JWT), сесія містить `globalRole` + `memberships[]`
Storage	S3-сумісне (`@aws-sdk/client-s3`), presigned URLs; локально — MinIO
Черги	BullMQ + ioredis (workers/)
Email	Resend (prod) / SMTP-MailHog (dev), no-op без ключа
Cron	in-process `node-cron` (`src/instrumentation.ts`), TZ Europe/Kyiv
Деплой	Railway (Dockerfile, auto-deploy на push у `main`)

Структура проєкту (коротко)

```
src/
├── app/
│   ├── (app)/          # авторизована зона (dashboard, standards, meetings, tasks, working-group
│   ├── (auth)/         # публічна зона (login, invite)
│   └── api/            # REST routes (auth, trpc, health, db-status, version, cron×3, meetings,
├── server/
│   ├── routers/        # 16 tRPC роутерів: user, workingGroup, standard, document, vote, meeting
│   │                   #   notification, dashboard, activityLog, comment, search, admin, suggest
│   │                   #   inlineComment, permission
│   ├── auth.ts         # NextAuth config (authorize/jwt/session callbacks)
│   ├── trpc.ts         # tRPC init + protectedProcedure + permissions middleware
│   ├── audit.ts        # logActivity() helper
│   ├── notify.ts, email.ts, s3.ts, ai/protocol.ts
├── lib/                # rbac.ts, env.ts, cron-jobs.ts, sanitizeHtml.ts, dueDate.tsx, toast.tsx,
│                       #   useLocalStorageState.ts, useExpandedTasks.ts, wordDiff.ts, ranks.ts,
│                       #   docxToHtml.ts, htmlToDocx.ts, ical.ts, __tests__/ (unit)
├── components/         # ui/ (Modal, ConfirmModal, DescriptionNote, Avatar, PageHeader…),
│                       #   layout/ (Sidebar, TopBar), auth/, dashboard/, standards/, meetings/,
│                       #   providers/ (ThemeProvider)
└── middleware.ts       # route gate (auth)
prisma/                 # schema.prisma (882 рядки), seed.ts, indexes.sql (migrations/ НЕМАЄ — AD
scripts/                # audit-coverage.ts, test-modules.ts, qa-smoke.sh, pre-db-push.{sh,sql}
workers/                # BullMQ воркери
QA-tests/               # архів QA-прогонів (див. QA-tests/README.md)
docs/                   # ARCHITECTURE.md (поточний стан), DECISIONS.md (ADR),
                        #   SPEC-document-editor.md, PORTABLE-SPEC-…md
```

> ⚠️ Одноразові скрипти (разові прогони проти прода з реальними даними) **не комітити** — репозиторій публічний, а історію git без force-push не вичистити.

Як запустити локально

Вимоги: Node 20+, pnpm 9, Docker. **Тримай репо ПОЗА iCloud** (`~/Documents` / `~/Desktop`) — iCloud-sync блокує `read()` на `node_modules` і вішає tsc/eslint/next build (див. `CONTINUATION.md §0`).

```
pnpm install
pnpm exec prisma generate            # ПЕРЕД typecheck/lint, інакше tsc сипле implicit-any
cp .env.example .env                 # заповнити NEXTAUTH_SECRET (openssl rand -base64 32)
docker compose up -d                 # postgres + redis + minio + mailhog
pnpm prisma db push                  # синк схеми (НЕ migrate dev — migration-файлів немає)
pnpm prisma db seed                  # admin@test.ua / Admin123! + тестові юзери
pnpm dev                             # http://localhost:3000
pnpm worker                          # (опційно) BullMQ воркери; або `pnpm dev:all`
```

Очікувані порти: app `3000`, postgres `5432`, redis `6379`, MinIO `9000`/console `9001` (minioadmin/minioadmin), MailHog SMTP `1025`/UI `8025`, Prisma Studio `5555` (`pnpm prisma:studio`).

> ⚠️ **Локальний gotcha (машина Kir):** порти 3000/5432/6379/9000/9001/1025/8025 можуть бути зайняті паралельним проєктом `buildco-platform`. Тоді: підняти інфру на зміщених портах через локальний (untracked, у `.gitignore`) `docker-compose.override.yml` з тегом `ports: !override` і запустити app на вільному порту (`PORT=3001 pnpm dev`), виставивши `NEXTAUTH_URL` / `NEXT_PUBLIC_APP_URL` на той самий порт.

Як прогнати тести

```
pnpm test                  # Vitest unit — 48 кейсів, 5 файлів у src/lib/__tests__/
pnpm test:coverage         # + HTML-звіт у ./coverage/
pnpm test:audit-coverage   # ts-morph: кожна tRPC-мутація має logActivity(); checklist*-процедури — у exempt-списку (потрібна жива БД)
pnpm test:modules          # integration CRUD по всіх сутностях через appRouter.createCaller (потрібна dev/test БД!)
pnpm typecheck && pnpm lint && pnpm build   # pre-push gate (ESLint --max-warnings 0)
bash scripts/qa-smoke.sh   # read-only smoke проти staging: NextAuth логін + ~12 перевірок
```

Production / Staging

- **Staging/prod URL:** https://standart.202ok.online/ (а також Railway-домен у `CONTINUATION.md §1`).
- **Repo:** github.com/kGuliaiev/standardotvoryets — `main` → Railway auto-deploy.
- **Логін для QA:** `admin@test.ua` (пароль — у seed `prisma/seed.ts` / `CONTINUATION.md`, тут не дублюємо).
- **Секрети/ENV:** НЕ зберігаються в репо. Дивись `.env.example` (контракт) + Railway → Variables (реальні значення). `.env` у `.gitignore`.

Lifecycle моделі (читати ОБОВ'ЯЗКОВО перед роботою з документами/голосуваннями)

Standard.status — машина станів (`STATUS_TRANSITIONS` у `src/server/routers/standard.ts`):

```
DRAFT  ──→  IN_REVIEW  ──→  VOTING  ──→  ADOPTED  ──→  ARCHIVED
  ↑           ↑    ↓                  ↘
  └───────────┘    └→ ARCHIVED          REJECTED → DRAFT (через vote.closeVoting)
```

- `VOTING` досягається ТІЛЬКИ через `vote.openVoting` ; `ADOPTED` / `REJECTED` — ТІЛЬКИ через `vote.closeVoting` (не через `standard.changeStatus`).
- `DRAFT → IN_REVIEW` потребує ≥1 активного `STANDARD` -документа (НЕ ТЗ). ADMIN — bypass.
- Поза voting flow: `IN_REVIEW → DRAFT/ARCHIVED` , `REJECTED → DRAFT/ARCHIVED` , `ADOPTED → ARCHIVED` , `ARCHIVED` — кінцевий.
- ADMIN bypass'ить state machine скрізь.

Document.lockedAt + Voting flow:

- Активний `STANDARD` -документ — це той, що `type='STANDARD'` + `lockedAt IS NULL` . Обмеження «1 STANDARD + 1 TECH_SPEC на стандарт» рахується ТІЛЬКИ серед unlocked.
- На `vote.closeVoting` / `closeOverdue` : знаходимо unlocked STANDARD-доку, виставляємо `lockedAt` , `lockedByVotingId` , `isCurrent=false` , дописуємо у `filename` суфікс `(Голосування №N, прийнято/відхилено DD.MM.YYYY)` , прив'язуємо `Voting.documentId = doc.id` .
- **REJECTED:** `standard.status = DRAFT` (не REJECTED!) + клонуємо документ у новий editable з `version = bumpVersion(prev)` (`v1 → v2`). РГ продовжує.
- **ADOPTED:** `standard.status = ADOPTED` . Новий клон НЕ створюється — заблокований снапшот і є фінал.
- Locked-документи — read-only для всіх, включно з ADMIN. `StandardBodyEditor` приймає `documentLocked` + `documentLockInfo` (seqNumber + closedAt) → банер + edit-blocked. Сервер (suggestion, document.update/setAsCurrent/updateMeta) теж відмовляє.
- Голосування **не видаляються**. Emergency cleanup — `vote.adminWipeAll` (ADMIN-only, type-to-confirm `WIPE-ALL-VOTINGS`) у `/admin/settings` → «Небезпечна зона».

Voting quorum: `forVotes / eligibleAtClose > 0.5` , де `eligibleAtClose` = snapshot активних `LEADER/DEPUTY/MEMBER` на момент закриття (SECRETARY і GUEST не голосують). Для архівних votings без snapshot — UI fallback на поточний `eligibleVoters` або `(for+against)` .

`DocumentType` (поточний enum): `STANDARD` | `TECH_SPEC` | `FEEDBACK` | `AGENDA` | `ATTACHMENT` (+ legacy `MEETING_MINUTES` для історичних рядків, але не в upload picker). Видалено: `DRAFT_STANDARD` (→ `STANDARD`), `FINAL` (→ `ATTACHMENT`) — мігровано через `scripts/pre-db-push.sql` (ADR-0005). `isCurrent` працює тільки для `STANDARD` / `TECH_SPEC` .

IN_REVIEW body lock (Pack 1)

Коли `Standard.status === 'IN_REVIEW'`, прямий WYSIWYG-edit body (тіла стандарту і `bodyHtml` прикріплених документів) заборонений для всіх крім ADMIN. Користувачі вносять зміни через Suggestions. Реалізовано через проп `standardStatus` у `StandardBodyEditor` + перевірки `lockedAt` у suggestion router. Не плутати з voting-lock (це окрема, абсолютна заборона після закриття голосування).

Модуль «Завдання» (поточний стан)

- **Задача** (`Task`) — статус/пріоритет/виконавець/дедлайн, прив'язана до стандарту, веде `ActivityLog`.
- **Підзадача** (`TaskChecklistItem`) — рівно один рівень вкладеності, поля як у задачі (`title` / `description` / `isDone` / `dueDate` / `assigneeId` / `order`), але **без** статус-машини, пріоритету й власного аудиту. Не враховується в KPI дашборда й у лічильниках простроченого (ADR-0007).
- **Шаблон** — `Standard.isTaskTemplate`. `task.copyFromTemplate` клонує задачі з підзадачами й **перераховує** дедлайни: зсув у днях від найближчого етапу джерела → той самий зсув від відповідного етапу цілі. Виконавці скидаються. Режими `append` / `replace` (ADR-0008).
- **Списки** — `/tasks` (дерево РГ → Стандарт або плоский список) і вкладка «Завдання» стандарту. Розгортання підзадач і режим перегляду зберігаються per-user у localStorage.

Що зараз у роботі

→ `CONTINUATION.md` (живий документ): поточна гілка, останній коміт, що закрито/відкрито, блокери, контекст архітектурних рішень. Повний опис архітектури й функціоналу — `docs/ARCHITECTURE.md`. Останній QA-цикл — `QA-tests/2026-05-26/`.

⛔ Що НЕ робити без узгодження з Kir

- **Не міняти Prisma schema** наосліп: Railway start-команда робить `prisma db push --accept-data-loss` на КОЖНОМУ деплої → необережна зміна = втрата даних на проді. Зміни схеми обговорювати окремо. Деструктивні зміни enum'ів — через boot-time backfill у `scripts/pre-db-push.sql` (див. ADR-0005).
- **Не перейменовувати ENV-змінні** — вони прив'язані в Railway Variables через reference (`${{Postgres.DATABASE_URL}}` тощо). Перейменування ламає прод-деплой.
- **Не force-push у `main`**, не комітити `.env`/секрети.
- **Не міняти базові порти** в `docker-compose.yml` (тільки локальний override).
- **Не чіпати UI 1:1 макети** без узгодження (Kir звіряє з прототипом).

Куди дивитись при проблемах (typical issues)

| Симптом | Причина / рішення | | ---------- | ---------- | | `tsc` / `eslint` / `build` висне на 0% CPU | репо в iCloud → перенести поза `~/Documents` / `~/Desktop` | | tsc: implicit-any / eslint no-unsafe флуд | не згенеровано Prisma client → `pnpm exec prisma generate` | | ESLint блокує деплой | `--max-warnings 0`: unused imports, `| |` замість `??`, hardcoded `bg-white` / `slate-\*` замість токенів | | Redirect loop / auth fail на проді | див. `OPS.md` (Cloudflare, NEXTAUTH_URL, cookie domain) | | Темна тема не перемикається на екрані | hardcoded `bg-slate-*` замість токенів (`bg-card` / `text-ink` / `text-mid` / `border-hairline` / `bg-page` / `bg-pill`) | | S3 upload падає локально | MinIO не піднятий / bucket не створений (`docker compose up -d minio`) | | Будь-яка прод-проблема | спершу `OPS.md` (operations runbook) |